

## ASSIGNMENT 3 — BRIDGE PATTERN

Name: Smat Zhangir

Group: SE-2522

Topic: A - Shape Rendering

Repository: <https://github.com/smatD/Assignment3>

Base commit: 417058ff0dda5163032b4ed1c10f14a3bd4addcc

### 1. Problem and Design

The assignment implements the Bridge design pattern for a simple shape-rendering application. The two dimensions that must vary independently are:

Shape abstraction - the domain objects being rendered:

- `Circle`, with a fixed radius of 2.
- `Square`, with a fixed side length of 3.

Rendering implementation - the mechanism used to render each shape:

- `VectorRenderer`
- `RasterRenderer`
- `AsciiRenderer` (the extension)

The Bridge pattern is appropriate because a shape should not contain or depend directly on a particular rendering implementation. Instead, `Shape` stores a reference to the `Renderer` interface and delegates rendering operations to it.

This allows the number of shapes and the number of rendering implementations to grow independently. For example, adding `AsciiRenderer` does not require creating `CircleAscii`, `SquareAscii`, or other combination subclasses.

### 2. Structure of the Bridge

The abstraction side consists of the abstract class `Shape` and its subclasses `Circle` and `Square`.

`Shape` stores the common domain identifier and a reference to the implementation:

```
private final String id;
private Renderer renderer;
```

The implementation side is represented by the `Renderer` interface:

```
public interface Renderer {
    String renderCircle(int radius);
    String renderSquare(int side);
}
```

`VectorRenderer`, `RasterRenderer`, and `AsciiRenderer` implement this interface.

The bridge is therefore the `Renderer` reference held by `Shape`. The implementation is supplied through the constructor:

```
protected Shape(String id, Renderer renderer) {
    this.id = id;
    this.renderer = renderer;
}
```

The implementation can also be changed later through:

```
public final void setImplementation(Renderer renderer)
```

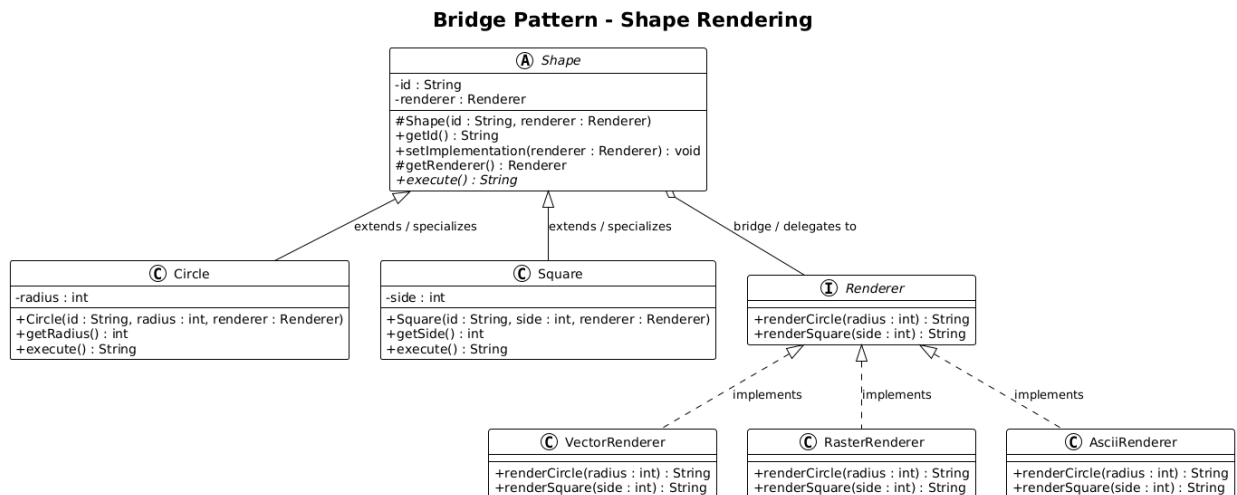
The public operation of the abstraction is `execute()`. Each concrete shape delegates this operation to the renderer without knowing which concrete renderer is being used.

For example, `Circle` contains:

```
@Override
public String execute() {
    return getRenderer().renderCircle(radius);
}
```

This demonstrates the essential Bridge relationship: the abstraction uses the implementation interface rather than constructing or inspecting a concrete implementation.

### 3. UML



The final design can be represented as two independently varying hierarchies connected by the Bridge.

- 'Shape' is the abstraction.
- 'Circle' and 'Square' are refined abstractions.
- 'Renderer' is the implementor interface.
- 'VectorRenderer', 'RasterRenderer', and 'AsciiRenderer' are concrete implementors.
- The association from 'Shape' to 'Renderer' is the bridge.

### 4. Clean-Code Decisions

#### 4.1 Program to an interface

'Shape' depends on 'Renderer' rather than a concrete renderer:

```
private Renderer renderer;
```

This keeps the abstraction independent of the implementation hierarchy. A new renderer can be introduced without modifying 'Shape', 'Circle', or 'Square'.

#### 4.2 Delegation instead of type checking

The shape does not contain conditions such as:

```

if (renderer instanceof VectorRenderer) {
    ...
}

```

Instead, it simply delegates:

```

return getRenderer().renderCircle(radius);

```

This keeps the code polymorphic and avoids coupling the abstraction to concrete implementations.

### 4.3 Constructor injection

The renderer is supplied to the shape through the constructor:

```

protected Shape(String id, Renderer renderer) {
    this.id = id;
    this.renderer = renderer;
}

```

This makes the dependency explicit and ensures that a `Shape` has an implementation when it is created.

### 4.4 Runtime switching

The implementation can be changed without creating a new shape:

```

switchable.setImplementation(new RasterRenderer());

```

The same `Circle` object is first executed using `VectorRenderer` and then using `RasterRenderer`.

The T5 test verifies this using reference equality:

```

originalReference == afterReference

```

It also checks that the shape's ID and radius remain unchanged.

#### 4.5 Small and focused renderer classes

Each concrete renderer implements the same small 'Renderer' interface and only performs rendering-specific behavior.

For example:

```
public final class AsciiRenderer implements Renderer {  
    @Override  
    public String renderCircle(int radius) {  
        return "ASCII circle radius=" + radius;  
    }  
  
    @Override  
    public String renderSquare(int side) {  
        return "ASCII square side=" + side;  
    }  
}
```

The renderer does not know about the 'Shape' object itself. It receives only the domain data necessary for rendering.

#### 5. Extension with I3

The third implementation, 'AsciiRenderer', was added after the base implementation was completed.

The important requirement of the extension is that the existing abstraction hierarchy does not need to be modified. 'Shape', 'Circle', and 'Square' remain unchanged.

The new class implements the existing 'Renderer' interface:

```
public final class AsciiRenderer implements Renderer {  
    @Override  
    public String renderCircle(int radius) {  
        return "ASCII circle radius=" + radius;  
    }  
}
```

```

@Override
public String renderSquare(int side) {
    return "ASCII square side=" + side;
}
}

```

The only application-level changes needed for the extension are the new renderer and the demo/test code in `Main`.

This demonstrates the benefit of the Bridge pattern: the implementation hierarchy can be extended independently from the abstraction hierarchy.

The extension is documented in `extension.diff`, generated from the base commit using:

```
git diff BASE_COMMIT HEAD -- src > extension.diff
```

## 6. Bridge versus Adapter

Bridge and Adapter can look similar because both commonly use interfaces and composition, but they solve different problems.

Bridge is a structural design decision made to separate two dimensions that are expected to vary independently. In this application, shapes and rendering methods are independent dimensions.

Adapter is normally used to make an existing incompatible interface work with another interface expected by the client.

This project uses Bridge because the renderer hierarchy is intentionally separated from the shape hierarchy from the beginning. The goal is not to adapt an existing incompatible renderer API.

## 7. Test Results

The demonstration performs seven tests:

| Test | Combination             | Expected result |
|------|-------------------------|-----------------|
| T1   | Circle + VectorRenderer | PASS            |
| T2   | Circle + RasterRenderer | PASS            |

|    |                              |      |
|----|------------------------------|------|
| T3 | Square + VectorRenderer      | PASS |
| T4 | Square + RasterRenderer      | PASS |
| T5 | Same Circle, Vector - Raster | PASS |
| T6 | Circle + AsciiRenderer       | PASS |
| T7 | Square + AsciiRenderer       | PASS |

The final demonstration produces:

T1 PASS | Circle + VectorRenderer | result=VECTOR circle radius=2

T2 PASS | Circle + RasterRenderer | result=RASTER circle radius=2

T3 PASS | Square + VectorRenderer | result=VECTOR square side=3

T4 PASS | Square + RasterRenderer | result=RASTER square side=3

T5 PASS | sameObject=true | stateUnchanged=true | before=VECTOR circle radius=2 | after=RASTER circle radius=2

T6 PASS | Circle + AsciiRenderer | result=ASCII circle radius=2

T7 PASS | Square + AsciiRenderer | result=ASCII square side=3

SUMMARY: 7/7 PASS

The T5 result is particularly important because it demonstrates that changing the implementation does not create a different shape object. The same 'Circle' instance is reused while its renderer changes.

## 8. Conclusion

The project demonstrates the Bridge pattern by separating shape abstractions from rendering implementations. 'Circle' and 'Square' can vary independently from 'VectorRenderer', 'RasterRenderer', and 'AsciiRenderer'.

The abstraction communicates with the implementation only through the 'Renderer' interface. No concrete renderer construction, casts, or implementation-specific branching are required inside the shape hierarchy.

The addition of 'AsciiRenderer' demonstrates the extensibility of the design: a new rendering implementation can be introduced without modifying the existing

shape classes. Runtime implementation switching further demonstrates that the Bridge is a real object reference rather than merely a static class relationship.